[Trung Nguyen] Process Fundamentals, Memory Layout

1. Process Identifiers, fork/vfork Functions

1.1 What fork() Is

I wanted to understand how bash works. Looking through the documentation, everything starts with `fork()`.

`fork()` is a system call that creates a **child process** — an almost exact copy of the parent process that called it. After `fork()` executes, **two processes exist simultaneously**, and **both continue running from the line right after `fork()`**.

Let's try it right away:

```
#include <stdio.h>
#include <unistd.h>

int main(void) {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    } else if (pid == 0) {
        printf("Child: PID=%d, PPID=%d\n", getpid(), getppid());
```

```
    } else {
        printf("Parent: PID=%d, child=%d\n", getpid(), pid);
    }
    return 0;
}
```

```
$ ./a.out
Parent: PID=4801, child=4802
Child: PID=4802, PPID=4801
```

Observations:

- `fork()` returns **twice, with two different values**: 0 in the child, the child's PID in the parent. This is the only thing that distinguishes the two processes.
- The parent needs to know the child's PID (to wait for it, to send it signals). The child doesn't — it already has `getppid()`. Hence 0 goes to the child.
- Running it repeatedly, you'll see **the order of the two lines flip around**. The kernel makes no guarantee about which side runs first.

Additional notes:

- The two sides' variables are **completely independent** after `fork()`. If the child modifies `x`, the parent doesn't see it. The kernel uses copy-on-write (duplicating a page only when one side writes to it), but that's only an optimization — the semantics are still two separate copies.
- `fork()` returns -1 with `EAGAIN` (out of PIDs, or hitting `RLIMIT_NPROC`) or `ENOMEM`.

Then I tried: Reading further, I found `vfork()`, advertised as "faster than `fork()`". I swapped it straight in:

```
pid_t pid = vfork();   /* changed fork -> vfork */

if (pid == 0) {
    printf("Child: PID=%d\n", getpid());
} else {
    printf("Parent: PID=%d\n", getpid());
}
return 0;
```

```
$ ./a.out
Child: PID=4902
Parent: PID=4901
Segmentation fault (core dumped)
```

Same logic, only the function name changed, and it crashes. So how does `vfork()` differ from `fork()`?

1.2 fork() vs vfork()

`vfork()` also creates a child process, but with two harsh conditions:

	fork()	vfork()
Address space	Private copy (COW)	**Shared with the parent**
Parent after the call	Runs in parallel with the child	**Suspended** until the child calls `exec()` or `_exit()`
What the child may do	Anything	**Only `exec()` or `_exit()`**

Why the Code Above Crashes

The child **shares the stack** with the parent. When the child returns from `main()`, it destroys `main()`'s stack frame — the very frame **the parent will return into** when it wakes up. The parent wakes, finds the stack corrupted, and crashes.

List of **undefined behavior** in a `vfork()` child process:

- Modifying any variable other than the `pid_t` that receives the return value
- Returning from the function that called `vfork()`
- Calling `exit()` (it must be `_exit()`)

```
/* WRONG - every line below is UB */
if (vfork() == 0) {
    x = 10;             /* modifies the parent's variable */
    printf("child\n");  /* stdio modifies the shared buffer */
    return 0;           /* destroys the parent's stack frame */
}

/* CORRECT - the ONLY valid form */
if (vfork() == 0) {
    execlp("ls", "ls", NULL);
    _exit(127);   /* _exit, NOT exit or return */
}
```

Practical Conclusion

`vfork()` came about when `fork()` still genuinely copied all of memory — a pointless expense if the child was going to `exec` right afterward. With modern copy-on-write, `fork()` is nearly as fast.

So my takeaway is: don't use `vfork()`. If you truly need to optimize fork+exec, use `posix_spawn()` — safe and clearly specified. Knowing `vfork()` well enough to read old code is enough.

Then I tried: Back to `fork()`, I added a sleep in the child to observe things more clearly:

```
if (fork() == 0) {
    sleep(2);
    printf("Child: PID=%d, PPID=%d\n", getpid(), getppid());
    _exit(0);
}
printf("Parent: PID=%d\n", getpid());
return 0;   /* parent exits immediately */
```

```
Parent: PID=5001
$ Child: PID=5002, PPID=1     <- PPID is no longer 5001!
```

The PPID changed to 1. So what exactly are PID and PPID, and who holds them?

1.3 Process Identifiers

```
pid_t getpid(void);    /* one's own PID - always succeeds */
pid_t getppid(void);   /* the parent's PID */
```

PPID Can Change — Orphan Processes

A PID is immutable for the life of a process. **A PPID is not.**

When the parent dies before the child, the child becomes an **orphan** and is **adopted** — its PPID changes to the adoptive parent's PID. Traditionally the adoptive parent is `init` (PID 1), which is why the output above shows `PPID=1`.

Note: on distros using systemd, the adoptive parent is not necessarily PID 1. A process can register itself as an adoptive parent with `prctl(PR_SET_CHILD_SUBREAPER, 1)`, and systemd makes the user manager a subreaper. **Don't write code that assumes `getppid() == 1`** to detect orphanhood.

PIDs Get Reused

The kernel allocates PIDs sequentially, then wraps around when it hits the ceiling (`/proc/sys/kernel/pid_max`, typically 4194304 on 64-bit systems; 32768 on 32-bit).

```
/* DANGEROUS */
pid_t pid = fork();
/* ... the child has died and been reaped somewhere ... */
kill(pid, SIGTERM);   /* may kill a COMPLETELY DIFFERENT process */
```

A PID is not a stable handle. Only use a PID when you're certain that process still exists.

The shell doesn't run into this problem thanks to one mechanism: as long as the parent hasn't "reaped" the child, the child's PID is **not yet released** — because the kernel still keeps an entry in the process table. We'll meet this detail again right below.

Three Other Identifiers That Come Along

Besides PID/PPID, every process also has a PGID (process group) and an SID (session). They are **inherited across** `fork()` and **preserved across** `exec()` — the foundation of job control, discussed in its own section.

```
ps -eo pid,ppid,pgid,sid,comm   # view all four
cat /proc/PID/stat              # PID, PPID, PGID, SID and much more
```

Then I hit a problem: I wanted to log to a file instead of the screen, so I reran the first program with a redirect:

```
printf("before fork\n");
if (fork() == 0) {
    printf("Child\n");
    exit(0);
}
printf("Parent\n");
return 0;
```

```
$ ./a.out
before fork
Parent
Child

$ ./a.out > out.txt
$ cat out.txt
before fork
```

```
Parent
before fork     <- printed TWICE!
Child
```

The same program: correct when it goes to the terminal, wrong when redirected to a file. The problem lies in how the process terminates.

2. exit Functions and Process Termination (main's return vs. _exit/exit)

Three Ways to Terminate, Not Equivalent

Method	Nature	What it does
`_exit(status)`	**System call**	Terminates **immediately**. No buffer flushing, nothing else runs
`exit(status)`	**Library function**	Runs functions registered with `atexit()` / `on_exit()` in **reverse order**, **flushes all stdio buffers**, closes streams, then calls `_exit()`
`return status;` from main	—	Equivalent to `exit(status)`

Since C99, falling off the end of `main` without a return is also equivalent to `exit(0)`. Under C89 the status is undefined — always write an explicit return.

```
static void bye(void) { printf("atexit ran\n"); }

int main(void) {
    atexit(bye);
    printf("hello\n");
    _exit(0);   /* does NOT print "atexit ran", and "hello" may be lost too */
    /* exit(0); -> prints both */
}
```

Explaining the Double-Print Bug

`fork()` copies **everything**, including the **contents of the stdio buffer** that the parent has `printf`'d but not yet flushed. The child carries that copy along. When the child calls `exit()`, it flushes its copy — and that content gets printed **a second time**.

Why does it only show up under redirection? Because stdout switches buffering mode based on its destination:

Destination	Mode	Consequence
Terminal	**Line-buffered**	`\n` flushes immediately -> buffer empty at `fork()` -> nothing to reprint
Pipe / file	**Fully-buffered**	Content sits in the buffer -> gets copied into the child

This is the worst kind of bug: it works in testing, and breaks once you drop it into a pipeline.

Two Fixes

```
/* Fix 1 (recommended): the child branch always uses _exit() */
if (fork() == 0) {
    printf("Child\n");
    _exit(0);   /* doesn't flush the inherited buffer */
}

/* Fix 2: clean the buffers before branching */
fflush(NULL);   /* NULL = flush ALL streams */
if (fork() == 0) { ... exit(0); }
```

Rule: in the child branch after `fork()`, always `_exit()`. This is also why the standard idiom writes `_exit(127)` after a failed `execvp()` — and why `vfork()` mandates `_exit()`.

Only the Low 8 Bits Reach the Parent

```
exit(0);     /* parent receives 0 */
exit(1);     /* parent receives 1 */
exit(256);   /* parent receives 0 - 256 & 0xFF == 0 */
exit(-1);    /* parent receives 255 */
```

Conventional usage:

Value	Meaning

0	Success
1–125	Program-defined errors
126	Command found but not executable (no execute permission)
127	Command not found
128 + n	**Shell convention**: killed by signal n. Ctrl-C (SIGINT=2) -> 130

Note that 128 + n is a **shell** convention, not a kernel one — the shell computes it itself when interpreting the status the child returns.

What Happens When a Process Terminates

Regardless of how it terminates, the kernel:

- Closes all open file descriptors
- Frees memory and memory mappings
- Turns any surviving child processes into orphans, handing them to the adoptive parent
- Sends `SIGCHLD` to the parent process
- **Retains one entry in the process table** holding the PID and termination status

Then I hit a problem: After fixing the double-print bug, I ran the program in a loop creating many children. Checking from a second terminal:

```
$ ps -eo pid,ppid,stat,comm | grep Z
5101 5100 Z a.out <defunct>
5102 5100 Z a.out <defunct>
5103 5100 Z a.out <defunct>
```

The children called `_exit(0)` cleanly and there are still corpses. Try killing them:

```
$ kill -9 5101
$ ps -eo pid,stat | grep 5101
5101 Z     <- SIGKILL has no effect either
```

This is exactly the "process table entry" the kernel retains, described above. It exists so the parent can come collect the **termination status** — and if the parent never comes, the corpse stays there forever, occupying a PID and a process-table slot.

3. wait, waitpid, and waitid Functions

3.1 wait() — Reaping Child Processes

The process corpse at the end of the previous section is called a **zombie**: a process that has terminated but whose parent hasn't collected its termination status.

The kernel has already freed nearly all of its resources (memory, file descriptors, memory mappings), keeping only **one entry in the process table** holding the PID and termination status — because the parent has the right to ask at any time. That's why `SIGKILL` is useless: you can't kill something that's already dead. Only the parent collecting it can remove that entry.

```
#include <sys/wait.h>
pid_t wait(int *status);
```

A single call does three things at once:

1. **Blocks** until **any** child terminates
2. **Reaps** that child — removes the zombie entry, frees the PID
3. Returns the PID of the reaped child, writing the status into `*status`

If the child is **already** a zombie when you call, `wait()` returns immediately.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    if (fork() == 0) {
        sleep(2);
        _exit(3);
    }

    int status;
    pid_t w = wait(&status);
    printf("child %d finished, status=%d\n", w, WEXITSTATUS(status));
    return 0;
```

```
}
```

```
$ ./a.out
child 5201 finished, status=3     <- after 2 seconds, and no zombie left
```

Notes:

- `wait()` returning -1 with `errno == ECHILD` means **there are no children left to wait for**. This is the standard stopping condition of a reaping loop, not an error.
- **"Waiting for a child to finish" and "cleaning up the child's corpse" are the same operation.** UNIX deliberately merges them — so the parent can't forget to clean up if it cares about the result.
- Zombies are only dangerous for **long-lived** processes (shells, daemons, servers). A program that runs and exits immediately leaves its zombies orphaned, and the adoptive parent cleans up for it.

Reading status — Always Use the Macros

`status` is **not** the exit code. It's an integer that **packs together** several pieces of information, with an unspecified bit layout.

```
if      (WIFEXITED(status))   printf("exited, code = %d\n",  WEXITSTATUS(status));
else if (WIFSIGNALED(status)) printf("killed, signal = %d\n", WTERMSIG(status));
else if (WIFSTOPPED(status))  printf("stopped, signal = %d\n", WSTOPSIG(status));
```

The three groups are **mutually exclusive**; always check the `WIFxxx` predicate before reading the corresponding value. `WIFSTOPPED` / `WIFCONTINUED` in particular only appear when explicitly requested via flags (see 3.2).

This is where the shell computes `$?`:

```
int last_status;
if      (WIFEXITED(status))   last_status = WEXITSTATUS(status);
else if (WIFSIGNALED(status)) last_status = 128 + WTERMSIG(status);  /* shell convention */
```

Then I hit a problem: I added background execution (`&`) to my shell. Running `sleep 30 &` then `ls`:

```
minish$ sleep 30 &
minish$ ls
minish$                       <- prompt returns IMMEDIATELY, while ls hasn't printed anything
Makefile minish minish.c      <- ls prints afterward, out of order
```

The shell thought `ls` was done. In fact `wait()` reaped the wrong thing — but which thing?

3.2 waitpid() — Choosing Which Child, and Whether to Block

`wait()` has two properties, and for a shell **both are wrong**:

Property	Why it's wrong
Waits for **any** child	With several children at once -> reaps a background job by mistake, losing the foreground job's status
Always blocks	A background job needs the prompt back immediately

In the example above: `sleep 30 &` hasn't finished, but if it happens to finish before `ls`, `wait()` returns `sleep`'s PID — and the shell thinks `ls` has ended.

```
pid_t waitpid(pid_t pid, int *status, int options);
```

The `pid` argument — choosing the target:

Value	Meaning
`> 0`	Wait for **exactly** the child with this PID
`-1`	Wait for **any** child — equivalent to `wait()`
`0`	Wait for a child **in the same process group** as the caller
`< -1`	Wait for a child in **process group** `

The `options` argument:

Flag	Effect
`WNOHANG`	**Don't block.** If no child has changed state -> return 0 immediately
`WUNTRACED`	Also return when a child is **stopped** — needed for `WIFSTOPPED`
`WCONTINUED`	Also return when a child is **continued** — needed for `WIFCONTINUED`

Return value — 0 is the new case compared to `wait()`:

Return	Meaning
`> 0`	PID of the child that just changed state

Return	Meaning
`0`	**Only with** `WNOHANG`: children exist, but none has changed state
`-1` + `ECHILD`	No matching children

Equivalence: `wait(&st)` ≡ `waitpid(-1, &st, 0)`.

Two Patterns Used in a Shell

```
/* Foreground job: block, specify the PID -> never a mix-up */
waitpid(pid, &status, 0);

/* Background job: poll, don't block */
for (int i = 0; i < njobs; i++) {
    pid_t w = waitpid(jobs[i].pid, &status, WNOHANG);
    if (w == 0) continue;                          /* still running */
    if (w  > 0) printf("[%d] Done\n", i + 1);      /* finished -> report and remove */
    jobs[i].pid = 0;
}
```

Notes:

- `WUNTRACED` is mandatory for job control. Without it, `waitpid()` only returns when the child **dies**, so Ctrl-Z would leave the shell hanging forever instead of printing `Stopped`.
- The `pid < -1` form waits for an entire pipeline with a single call — useful once each job has its own process group.

waitid()

```
int waitid(idtype_t idtype, id_t id, siginfo_t *infop, int options);
```

It differs from `waitpid()` in four ways:

- **Splits target selection into two arguments**: `idtype` (`P_PID`, `P_PGID`, `P_ALL`) and `id` — no more encoding it with a negative sign.
- **Assumes nothing**: you must explicitly state `WEXITED`, `WSTOPPED`, or `WCONTINUED`.
- **Returns results via** `siginfo_t`: read `si_pid`, `si_status`, `si_code` (`CLD_EXITED`, `CLD_KILLED`, `CLD_DUMPED`, `CLD_STOPPED`, `CLD_CONTINUED`) — no `WIF*` macros needed, and it can even distinguish the core-dump case.
- **Has** `WNOWAIT`: retrieve the information but **leave the child in a still-waitable state** — poll now, reap later.

`waitpid()` is still the default choice because it's more compact. Knowing `waitid()` exists is enough.

Then I hit a problem: Background jobs only got reaped when I pressed Enter. Leave the prompt idle for 10 minutes and the zombies just sat there. I switched to using `SIGCHLD` so the kernel would notify the shell:

```
void sigchld_handler(int sig) {
    int status;
    waitpid(-1, &status, WNOHANG);   /* reap one child */
}
```

Testing by creating 3 children that die almost simultaneously:

```
$ ps -eo pid,stat | grep Z
5301 Z     <- still 2 zombies
5302 Z
```

The handler ran, yet zombies remain. This is the first bug in the race-condition family.

4. Race Conditions Between fork/exec/wait

This family of bugs shares one characteristic: **it works nine times and fails the tenth**. The cause isn't in the code but in the execution order the kernel decides.

The root cause in one sentence: **after `fork()`, the run order of parent and child is indeterminate.**

Race 1 — SIGCHLD Doesn't Queue

Standard signals **are not counted**. Three children dying almost simultaneously -> the parent may receive only **one** `SIGCHLD`. The handler reaps exactly one child and returns -> the other two zombies stay forever.

```
/* WRONG */
void handler(int sig) {
    waitpid(-1, NULL, WNOHANG);
}

/* CORRECT - loop until there's nothing left */
```

```
void handler(int sig) {
    int saved = errno;   /* the handler may clobber errno */
    while (waitpid(-1, NULL, WNOHANG) > 0)
        ;
    errno = saved;
}
```

The stopping condition is `<= 0`: 0 = children exist but none is done; -1 + `ECHILD` = no children left.

Race 2 — EINTR

Once the handler finishes, a blocking system call may return -1 with `errno == EINTR` instead of continuing.

```
/* WRONG - the REPL loop exits by mistake when SIGCHLD arrives */
if (fgets(line, sizeof line, stdin) == NULL) break;

/* CORRECT - distinguish real EOF from being interrupted */
while (fgets(line, sizeof line, stdin) == NULL) {
    if (errno == EINTR) { clearerr(stdin); continue; }
    goto done;   /* real EOF (Ctrl-D) */
}
```

Another approach: set the `SA_RESTART` flag when installing the handler with `sigaction()`. Note that `SA_RESTART` **cannot restart every system call** — some still return `EINTR` regardless.

Race 3 — Non-async-signal-safe Functions in a Handler

The handler runs interleaved between **any two instructions** of the main program, including partway through a library function. Calling `printf()` or `malloc()` inside a handler can corrupt their internal state.

```
/* WRONG */
void handler(int sig) {
    printf("child is done\n");   /* printf is NOT async-signal-safe */
}

/* CORRECT - the handler only sets a flag; the main program does the real work */
```

```
volatile sig_atomic_t child_done = 0;

void handler(int sig) { child_done = 1; }

/* in the main loop */
if (child_done) { child_done = 0; reap_jobs(); }
```

`write()` and `waitpid()` are async-signal-safe; `printf()` and `malloc()` are not.

Race 4 — Operations That Must Complete Before the Child exec()s

The shell needs to set the child's process group **before** the child execs (after exec, `setpgid()` returns `EACCES`). But:

- If only the parent does it -> the child may exec first -> `EACCES`
- If only the child does it -> the parent may send a signal before the child joins the group

Solution: both do it, with the same arguments, and whichever arrives second ignores the error.

```
pid_t pid = fork();

if (pid == 0) {
    setpgid(0, pgid);        /* the child does it itself */
    execvp(argv[0], argv);
    _exit(127);
}
if (setpgid(pid, pgid) == -1 && errno != EACCES)   /* the parent does it too */
    perror("setpgid");
```

The operation is idempotent, so a duplicate call is harmless. **`EACCES` on the parent's side is not an error** — it's the signal "the child has already exec'd and did it itself."

This pattern applies to **every** operation that must complete before exec.

Race 5 — Not Knowing Whether exec Succeeded or Failed

The parent only knows that `fork()` succeeded. The child has already transformed and can't be asked.

```
int pfd[2];
pipe2(pfd, O_CLOEXEC);   /* the key: O_CLOEXEC */

pid_t pid = fork();
if (pid == 0) {
    close(pfd[0]);
    execvp(argv[0], argv);
    int e = errno;
    write(pfd[1], &e, sizeof e);   /* exec failed -> report errno to the parent */
    _exit(127);
}

close(pfd[1]);
int e, n = read(pfd[0], &e, sizeof e);
close(pfd[0]);

if (n == 0) { /* EOF -> exec SUCCEEDED (the kernel closed the pipe itself) */ }
else        { fprintf(stderr, "%s: %s\n", argv[0], strerror(e)); }
```

The idea: **a successful exec becomes the signal by itself** — the child has to do nothing at all.

Race 6 — stdio Buffers (Already Seen in 2)

`fflush(NULL)` before `fork()`, or `_exit()` in the child branch.

Then I hit a problem: Having dealt with the pile of races, I noticed the C library already had a `system()` function that does everything in one line. I tried using it for a "run a command from a config file" feature:

```
char cmd[256];
snprintf(cmd, sizeof cmd, "cat %s", filename);
system(cmd);
```

A user entered the filename `a.txt; rm -rf ~`. The home directory vanished.

5. system Function

```
int system(const char *command);
```

`system()` isn't a new mechanism. It's exactly the three things we just built, glued together:

```
fork()
  -> child:  execl("/bin/sh", "sh", "-c", command, NULL)
  -> parent: waitpid(pid, &status, 0)
  -> returns the status word
```

The crux: **it inserts an extra `/bin/sh` process in the middle.** Every advantage and every problem follows from that one decision.

The advantage: you get the full shell syntax for free — pipes, redirection, globs, `&&`.

```
system("ls *.c | wc -l > out.txt");   /* works, nothing to implement */
```

Four Problems

1. Command injection — the most serious, and exactly the bug above.

The string is **interpreted** by `/bin/sh`, so every special character in user data becomes code. Manual escaping isn't a solution: the set of characters needing escapes differs between shells and between quoting contexts.

```
/* CORRECT - fork + execvp is completely immune */
char *args[] = { "cat", filename, NULL };
if (fork() == 0) { execvp(args[0], args); _exit(127); }
wait(NULL);
```

`argv[]` is an array of strings with **no interpretation step whatsoever**. A filename containing a semicolon is just a weird filename.

2. Environment dependence. `/bin/sh` reads `PATH` and `IFS` from an environment the user controls. Poison `PATH` -> `system("cp ...")` runs a fake `cp`. For this reason, **`system()` must never be used in a set-user-ID program**.

3. Cost. Each call creates **two** processes instead of one, plus the shell's parsing overhead.

4. Three-tiered return value:

Return	Meaning
-1	Couldn't create the child, or `waitpid` failed

Return	Meaning
127 (as `WEXITSTATUS`)	The shell couldn't run the command — **or** that command itself returned 127
other	The **shell's** status word, unpacked with `WIFEXITED` / `WEXITSTATUS`

The annoying part: you receive `/bin/sh`'s status, **not that of the command you wanted to run**. If the shell is killed by a signal, you have no idea what happened to the command inside.

`system(NULL)` returns nonzero if the system **has** a shell — that's how you test for a shell's existence, not how you run a command.

Conclusion

Our shell doesn't use `system()` for two reasons:

- It inserts an intermediate shell that our own program is trying to **become**.
- It throws away **the gap between fork and exec** — the place where we need to set the process group, restore signal dispositions, rearrange file descriptors, and set resource limits.

`system()` is still reasonable for utility scripts whose input you write entirely yourself. The moment external data enters the command string, switch to fork + execvp.

Next problem: The shell now runs commands, reports `$?`, and runs background jobs. But pressing Ctrl-C **kills the shell too**, and a background job `cat &` fights the foreground command over the keyboard.

6. Process Groups, Sessions, Controlling Terminal & Job Control

6.1 Process Group — A Label for Signalling a Whole Group

The current shell runs `find / | grep foo | wc -l`, then you press Ctrl-C:

```
minish$ find / | grep foo | wc -l
^C
$      <- minish dies TOO, back to bash
```

The terminal driver only sees one 0x03 character and has to decide who to send `SIGINT` to. Right now it sends it to **everything** attached to the terminal — including the shell.

The naive approach is for the shell to track the PIDs of all three processes and signal each one. It breaks immediately: if `grep` forks something of its own, the shell **has no idea**, and that grandchild process survives.

We need a **kernel-managed label, automatically inherited across `fork()`**, that lets you signal a whole group with one call. That's the **process group** (PGID). The shell term **job** *is* a process group.

```
pid_t getpgrp(void);                  /* one's own PGID */
pid_t getpgid(pid_t pid);
int   setpgid(pid_t pid, pid_t pgid); /* setpgid(0,0) = create a new group for myself */
int   killpg(pid_t pgid, int sig);    /* = kill(-pgid, sig) */
```

```
#include <stdio.h>
#include <unistd.h>

int main(void) {
    printf("before: PID=%d PGID=%d\n", getpid(), getpgrp());
    setpgid(0, 0);   /* create a new group, become its leader */
    printf("after:  PID=%d PGID=%d\n", getpid(), getpgrp());
    return 0;
}
```

```
before: PID=5401 PGID=5300     <- inherited the shell's PGID
after:  PID=5401 PGID=5401     <- PGID == PID -> I'm the leader
```

Notes:

- **A group survives after its leader dies.** Its lifetime extends until the last member leaves (dies or moves to another group).
- The kernel guarantees a new PID **won't collide with the PGID/SID of an existing process** — nobody accidentally becomes leader of a live group.
- Recall `waitpid()` with `pid < -1`: wait for any child in a given process group. Only now is it usable.

Four Constraints on setpgid()

Violation	Error
`pid` is neither the caller nor a **direct child**	`ESRCH`
Moving into a group belonging to a **different session**	`EPERM`
`pid` is a **session leader**	`EPERM`
The child has **already exec()'d** before the change	`EACCES`

That last constraint is exactly Race 4 from section 4 — and now it's clear why it exists: a running program would be confused if its PGID changed out from under it.

Then I hit a problem: I put each job in its own group, and made the shell ignore `SIGINT` so Ctrl-C wouldn't kill it:

```
signal(SIGINT, SIG_IGN);   /* in the shell */
...
if (pid == 0) { setpgid(0, 0); execvp(argv[0], argv); _exit(127); }
setpgid(pid, pid);
```

```
minish$ sleep 100
^C          <- nothing happens
^C^C^C      <- sleep won't die
```

Exactly the opposite: now the shell survives but the job is **immune** to Ctrl-C.

6.2 SIG_IGN Is Inherited Across exec — The Asymmetric Trap

Recalling the table from section 1.1:

Disposition before exec	After exec
Custom handler	-> `SIG_DFL` (kernel resets it)
`SIG_IGN`	-> **still** `SIG_IGN`
`SIG_DFL`	-> `SIG_DFL`

The shell ignores `SIGINT` for itself. The forked child inherits `SIG_IGN`, and exec **doesn't clear it** -> `sleep` runs with `SIGINT` ignored.

```
/* CORRECT - the child restores SIG_DFL before exec */
if (pid == 0) {
    setpgid(0, 0);
    signal(SIGINT,  SIG_DFL);
    signal(SIGQUIT, SIG_DFL);
    signal(SIGTSTP, SIG_DFL);
    signal(SIGTTIN, SIG_DFL);
    signal(SIGTTOU, SIG_DFL);
    execvp(argv[0], argv);
    _exit(127);
}
```

The flip side of the same rule — applying to ordinary applications, not shells:

```
/* Only install a handler if the signal is NOT already ignored */
struct sigaction sa;
sigaction(SIGTSTP, NULL, &sa);
if (sa.sa_handler != SIG_IGN) {
    sa.sa_handler = my_handler;
    sigaction(SIGTSTP, &sa, NULL);
}
```

The reason: a shell without job-control support sets `SIGTSTP`/`SIGTTIN`/`SIGTTOU` to `SIG_IGN` **deliberately**; `nohup(1)` ignores `SIGHUP` deliberately. Overriding that breaks the user's intent.

Then I hit a problem: With that fixed, Ctrl-C kills the right job. But running `cat &` and then typing another command:

```
minish$ cat &
minish$ ls
-> the characters I type sometimes go to cat, sometimes to minish
```

Bash doesn't have this problem. Why?

6.3 Sessions and the Controlling Terminal

Process groups solve "who to send the signal to." "Who gets to read the keyboard" needs one more layer.

```
session          -- one login session / one terminal window
└─ process group -- one job
```

```
    └─ process
```

The boundary is **hard**: every member of a process group **must be in the same session**.

Every process in a session shares **one controlling terminal**. It is established when the **session leader** first `open()`s a terminal that doesn't already belong to another session (unless `O_NOCTTY` is used). A terminal belongs to **at most one session**. It's inherited across `fork()` and **preserved across `exec()`**.

A session leader that opens a terminal becomes the **controlling process** -> the kernel sends it `SIGHUP` when the terminal disconnects (window closed, SSH dropped).

Foreground Process Group — The Key

At any moment, a terminal has **exactly one foreground process group**:

- **Only the foreground group may read input.** A background process that `read()`s from the terminal receives **`SIGTTIN`** and **is stopped**. This is precisely why bash doesn't fight over stdin.
- **Signal-generating characters only reach the foreground group**: Ctrl-C -> `SIGINT`, Ctrl-\ -> `SIGQUIT`, Ctrl-Z -> `SIGTSTP`. The signal goes to **the entire group**, so every process in a pipeline receives it, including grandchildren the shell doesn't know about.
- A background process that `write()`s while the `TOSTOP` flag is set -> **`SIGTTOU`**.

```
pid_t tcgetpgrp(int fd);                /* the current foreground group */
int   tcsetpgrp(int fd, pid_t pgid);    /* change the foreground group */
pid_t getsid(pid_t pid);
pid_t setsid(void);
```

The full picture for `find / | wc -l &` and `sort < f | uniq -c`:

```
Session 400 ── controlling terminal (foreground PGID = 660)
├─ PG 400   bash  PID=400    <- session leader + controlling process
├─ PG 658   find  PID=658    <- leader ┐ background
│           wc    PID=659              ┘
└─ PG 660   sort  PID=660    <- leader ┐ FOREGROUND
            uniq  PID=661              ┘
```

Three takeaways:

- **All four have PPID 400** — the shell is the direct parent of every pipeline component, not a chain of parents and children.
- **A job's PGID = the PID of the first process** in the pipeline.
- The shell sits alone in its own group.

setsid() — Detaching From a Session

It does **three things at once**: becomes session leader of a new session; simultaneously becomes process group leader of a new group (PID == PGID == SID); and **loses its controlling terminal**.

It fails with `EPERM` if the calling process is already a process group leader — because if it were allowed, the leader would jump to a new session while its group members stayed in the old one, breaking the hierarchy.

```
/* Daemon pattern */
switch (fork()) {
case -1: perror("fork"); exit(1);
case  0: break;                     /* child: new PID -> definitely not a leader */
default: _exit(EXIT_SUCCESS);       /* parent exits */
}
if (setsid() == -1) { perror("setsid"); exit(1); }
```

To check whether a controlling terminal remains: `open("/dev/tty", O_RDWR)` — it fails with `ENXIO` if there isn't one.

Note: after `setsid()`, the process is a session leader with **no** controlling terminal, so it **may accidentally acquire** one on a later `open()`. Two ways to prevent this: always use `O_NOCTTY`, or `fork()` a second time (the grandchild isn't a session leader -> it can't acquire one). This is why classic daemons fork twice.

Then I hit a problem: I added `tcsetpgrp()` to hand the terminal to the foreground job, then take it back once the job finished:

```
tcsetpgrp(STDIN_FILENO, pid);       /* hand it to the job */
waitpid(pid, &status, 0);
tcsetpgrp(STDIN_FILENO, getpgrp());  /* take it back */
```

```
minish$ ls
Makefile minish
        <- hangs, no more prompt
$ jobs
[1]+  Stopped   ./minish     <- minish STOPPED itself
```

6.4 tcsetpgrp() Stabbing Itself With SIGTTOU

`tcsetpgrp()` is a **write** operation on the terminal. Once the shell hands the terminal to the job, the shell becomes **background**. The next call to **take the terminal back** comes from a background group -> the kernel sends `SIGTTOU` -> the shell stops itself.

```
/* Mandatory, right at shell startup */
signal(SIGTTOU, SIG_IGN);
signal(SIGTTIN, SIG_IGN);
signal(SIGTSTP, SIG_IGN);   /* so Ctrl-Z doesn't stop the shell */
signal(SIGINT,  SIG_IGN);
signal(SIGQUIT, SIG_IGN);
```

The complete shell-initialization pattern:

```
static pid_t shell_pgid;

void shell_init(void) {
    /* 1. Wait until the shell is genuinely in the foreground.
     *    If started in the background, send ourselves SIGTTIN to stop
     *    until the user brings us to the foreground. */
    while (tcgetpgrp(STDIN_FILENO) != (shell_pgid = getpgrp()))
        kill(-shell_pgid, SIGTTIN);

    /* 2. Ignore all job-control signals - SIGTTOU matters most */
    signal(SIGINT, SIG_IGN);  signal(SIGQUIT, SIG_IGN);
    signal(SIGTSTP, SIG_IGN); signal(SIGTTIN, SIG_IGN); signal(SIGTTOU, SIG_IGN);

    /* 3. Put the shell in its own process group */
    shell_pgid = getpid();
    setpgid(shell_pgid, shell_pgid);

    /* 4. Acquire the controlling terminal */
    tcsetpgrp(STDIN_FILENO, shell_pgid);
}
```

Note: `tcsetpgrp()` must also be called in **both parent and child**, just like `setpgid()` — for the same race reason from section 4. The child grabs the terminal before exec; the parent calls it too in case the child is slow.

fg and bg Differ by Exactly One Operation

	tcsetpgrp()	killpg(SIGCONT)	shell waits

fg	□ hands over the terminal	□	□
bg	□ keeps the terminal	□	□

"Foreground" **means only this**: this group currently holds the terminal, and the shell is waiting on it.

```
void do_fg(pid_t pgid) {
    tcsetpgrp(STDIN_FILENO, pgid);
    killpg(pgid, SIGCONT);
    int status;
    waitpid(-pgid, &status, WUNTRACED);   /* WUNTRACED: mandatory */
    tcsetpgrp(STDIN_FILENO, getpgrp());   /* take it back */
}

void do_bg(pid_t pgid) {
    killpg(pgid, SIGCONT);   /* just wake it up, don't hand over the terminal */
}
```

`WUNTRACED` **is mandatory.** Without it, `waitpid()` only returns when the child **dies** -> Ctrl-Z leaves the shell hanging forever instead of printing `Stopped`.

After every job, the shell **must take the terminal back**. Forget this step and no group is the foreground, so the shell's own `read()` gets stopped by `SIGTTIN` immediately.

Orphaned Process Groups

The permanently-stuck scenario: a child gets **stopped**, then the parent dies. The shell removes the parent's group from its job table; the adoptive parent only reaps zombies and doesn't care about stopped processes -> **nobody knows to send** `SIGCONT`.

Definition: a group is orphaned if **no** member has a parent that is in **the same session but a different process group**. It follows that **a session leader is always in an orphaned group**.

The kernel plugs the hole: when a group **becomes** orphaned **and has at least one stopped member**, the entire group receives `SIGHUP` followed by `SIGCONT`. If no member is stopped -> **no signal is sent**.

API behavior changes in an orphaned group:

Operation	Normal	Orphaned group
Background `read()` from the terminal	`SIGTTIN`	`read()` fails with `EIO`
Background `write()` with `TOSTOP`	`SIGTTOU`	`write()` fails with `EIO`

Operation	Normal	Orphaned group
`SIGTSTP` / `SIGTTIN` / `SIGTTOU` that would stop the process	stops it	**silently discarded** (unless being handled)
`tcsetpgrp()`	OK	`ENOTTY`

Then I hit a problem: Job control works fine. I added the `cd` and `export` built-ins. `cd` works, but `export`:

```
minish$ export MYVAR=hello
minish$ ./print_myvar
MYVAR = (null)     <- the child program doesn't see the variable
```

7. Command-Line Arguments and Environment List

`exec` **wipes out** the memory image — heap, stack, and globals all disappear. You can't pass data through memory.

Only **two channels** for passing data across exec remain, and that's why both exist:

Channel	Characteristics
argv	Explicit, **per invocation**. The user can see it — and so can every other user, via `ps`
environ	Implicit, **inherited across many generations of children**. Set `TZ` once and every descendant process gets it

(The other inherited things — fds, cwd, umask, PGID — are *process attributes*, not data.)

Why export Is Necessary

The bug above happened because minish stored `MYVAR` in its own private variable table, never putting it into `environ`. `execve()` only passes `environ` to the child.

This is exactly the distinction bash makes:

```
VAR=value        # just a shell variable - the child does NOT see it
export VAR       # put it in environ - the child sees it
```

```
VAR=value ./prog # put it in environ for THAT invocation only
```

API

```
extern char **environ;   /* array of "NAME=value", NULL-terminated */

char *getenv(const char *name);
int   setenv(const char *name, const char *value, int overwrite);
int   putenv(char *string);
int   unsetenv(const char *name);
char *secure_getenv(const char *name);
```

```
/* the export built-in in a shell */
if (strncmp(argv[0], "export", 6) == 0) {
    char *eq = strchr(argv[1], '=');
    if (eq) { *eq = '\0'; setenv(argv[1], eq + 1, 1); }
    continue;
}
```

Iterating over all of it:

```
extern char **environ;
for (char **ep = environ; *ep != NULL; ep++)
    puts(*ep);
```

Then I hit a problem: My shell needed a large buffer to read long command lines. I declared:

```
char cmdbuf[10 * 1024 * 1024];   /* 10 MB, a global variable */
```

After building, I checked the file size:

```
$ ls -l minish
-rwxr-xr-x 1 trung trung 17048 minish     <- only 17 KB?
```

I initialized it just to be sure:

```
char cmdbuf[10 * 1024 * 1024] = { 'x' };   /* added an initializer */
```

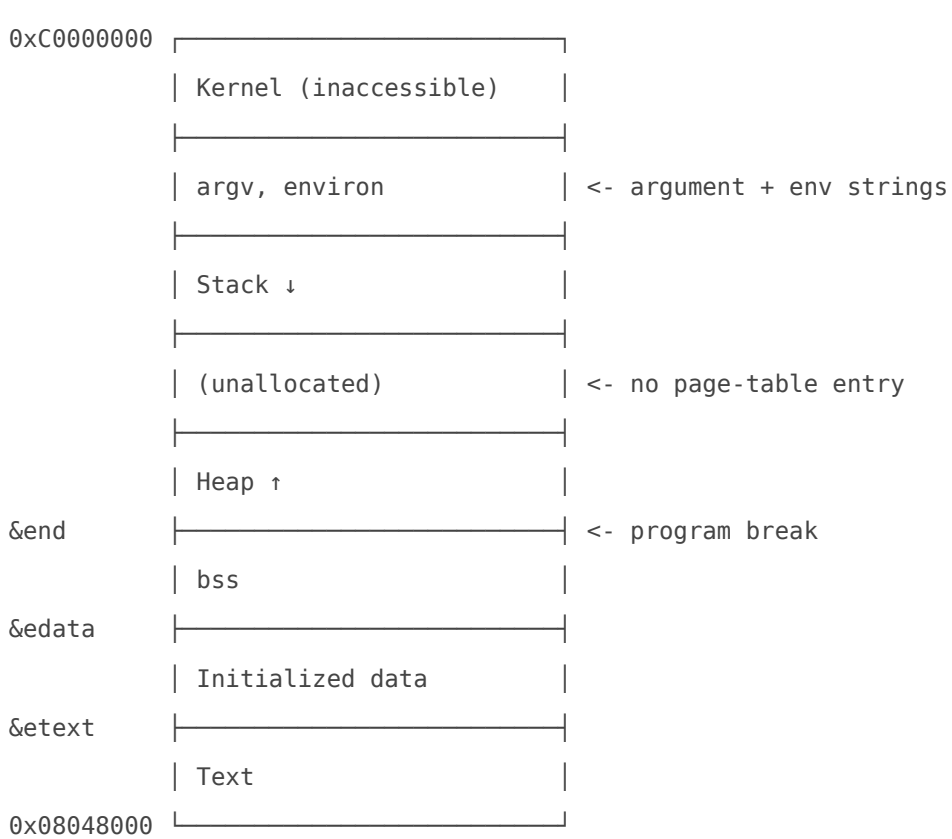
```
$ ls -l minish
-rwxr-xr-x 1 trung trung 10502104 minish   <- 10 MB!
```

The same array, just one initializer character added, and the file swelled from 17 KB to 10 MB.

8. Memory Layout of a C Program (text/data/bss/heap/stack)

Process memory is divided into **segments**:

Segment	What it holds	Properties
Text	Machine instructions	**Read-only**, **shareable** among all processes running the same program
Initialized data	Globals/statics **with an explicit initializer**	Values **read from the executable file** at load time
Uninitialized data (bss)	Globals/statics **without an initializer**	The system **zero-fills** it before execution
Stack	Stack frames: local variables, arguments, return addresses	**Grows downward**
Heap	Dynamic allocation at run time	**Grows upward**; its top is called the **program break**



Explaining the 17 KB -> 10 MB Bug

This is precisely why data and bss are split into two segments: **while the program sits on disk, there's no need to allocate space for uninitialized data.** The executable only records the *location and size required*; the real space is provided by the program loader at run time.

Adding `= { 'x' }` turns the array from bss into initialized data -> 10 MB of values that must be written into the file.

```
$ size ./minish
   text    data      bss      dec      hex  filename
   1234     620 10485760 10487614   a0043e  minish     <- big bss, small file
```

Rule: when declaring a large array, **don't initialize it** if all you need is zeros — the system zero-fills it already. In embedded work, this is the difference between "fits in flash" and "doesn't fit in flash."

32-bit and 64-bit: Architecture, Kernel, and Process

Three levels that must be distinguished

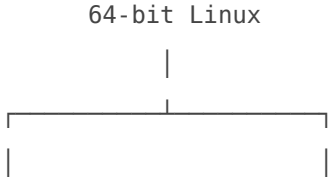
The term "32-bit" or "64-bit" can refer to three different things:

- **CPU architecture** — the hardware instruction set (e.g. x86-64)
- **Operating system / kernel** — the mode the kernel itself runs in
- **Process / application** — the ABI the program was built against

These levels are independent. A perfectly valid combination:

```
CPU:       x86-64
Kernel:    64-bit Linux
Program:   32-bit
```

A 64-bit system can run both 64-bit and 32-bit programs:



```
    64-bit process         32-bit process
          |                      |
      64-bit ABI             32-bit ABI
      64-bit pointers        32-bit pointers
```

Memory layout of a 32-bit process

```
Higher Address
0xFFFFFFFF
+------------------+
|      Kernel      |
+------------------+
|                  |
|      Stack       |
|        ↓         |
|                  |
|                  |
|        ↑         |
|      Heap        |
+------------------+
|       BSS        |
+------------------+
|       Data       |
+------------------+
|       Text       |
+------------------+
0x00000000
Lower Address
```

The virtual address is 32 bits wide, so the theoretical address space is:

```
2^32 = 4 GB
```

A 32-bit process therefore has a virtual address space of at most about 4 GB. On Linux, the portion actually available to user space is smaller and depends on the kernel, its configuration, and the architecture.

Memory layout of a 64-bit process

```
Higher Address
+------------------+
```

```
|      Stack       |
|        ↓         |
+------------------+
|                  |
|      mmap        |
|                  |
|        ↑         |
+------------------+
|       Heap       |
+------------------+
|       BSS        |
+------------------+
|       Data       |
+------------------+
|       Text       |
+------------------+
Lower Address
```

The segment structure is the same. The major difference is that addresses and pointers are wider.

Pointer width

```
int x = 10;
int *p = &x;
```

On a typical system:

```
32-bit:  sizeof(int *) == 4
64-bit:  sizeof(int *) == 8
```

A 32-bit process on a 64-bit Linux kernel

The process still uses the 32-bit ABI and still has a 32-bit virtual address space:

```
32-bit process


0x00000000
      |
      |   User virtual address space
      |
0xFFFFFFFF
```

It does not gain access to addresses such as:

```
0x00007xxxxxxxxxx
```

merely because the kernel is 64-bit.

```
64-bit OS
   │
   ├── 64-bit process
   │       └── 64-bit address space
   │
   └── 32-bit process
           └── 32-bit address space
```

Note: a 32-bit process running on a 64-bit Linux system generally uses the 32-bit ABI and a 32-bit virtual address space, but its exact memory mapping and layout are determined by the Linux kernel and the architecture, so it is not necessarily identical to that of a native 32-bit system.

Mapping C Variables -> Segments

```
char globBuf[65536];            /* bss */
int primes[] = { 2, 3, 5, 7 };  /* initialized data */

void f(int val)                 /* val: stack (f's frame) */
{
    static int key = 9973;      /* initialized data - NOT the stack */
    static char mbuf[102400];   /* bss */
    int local;                  /* stack, GARBAGE VALUE */
    char *p = malloc(1024);     /* p: stack; the region p points to: heap */
}
```

Four takeaways:

- **A `static` inside a function does not live on the stack** — it's in data/bss like a global, differing only in name scope.
- What decides data vs bss is **whether there's an explicit initializer**, not global vs static.
- **The pointer is on the stack, the region it points to is on the heap** — two different things.
- **Only bss is zero-filled.** The stack and heap contain **garbage** (`calloc` does give zeros).

Then I hit a problem: I split the command-line parser into its own library for reuse. Building it as a `.so`:

```
$ gcc -c parser.c
$ gcc -shared -o libparser.so parser.o
/usr/bin/ld: parser.o: relocation R_X86_64_32 against `.rodata' can not be used
when making a shared object; recompile with -fPIC
```

Adding `-fPIC`, it builds. But running it:

```
$ gcc minish.c -L. -lparser -o minish     <- build OK
$ ./minish
./minish: error while loading shared libraries: libparser.so:
cannot open shared object file: No such file or directory
```

9. Shared Libraries: Static vs. Dynamic Linking

The Core Difference

	Static (.a)	Shared (.so)
When code is linked	**At build time**	**At run time**, by the dynamic linker (`ld.so`)
Executable contents	Contains the library code itself	Only records **the names of the libraries needed**
Copies on disk	One per program	**A single copy**
Copies in RAM	One per process	**One shared copy**
Patching a library bug	Rebuild **every** program	Rebuild **one** `.so` file
Running on an unfamiliar machine	Always works	Depends on the `.so` being present, at the right version
Special compile flag	none	`-fPIC`

Static linking completes everything at build time; dynamic linking defers part of it to run time. Every advantage and disadvantage follows from that sentence.

```
# Static
gcc -c foo.c && ar rcs libmy.a foo.o
gcc main.c -L. -lmy -o app     # the code is COPIED into app
```

```
# Shared
gcc -fPIC -c foo.c
gcc -shared -Wl,-soname,libmy.so.1 -o libmy.so.1.0.0 foo.o
gcc main.c -L. -lmy -o app     # only records "needs libmy.so.1"
```

Explaining the -fPIC Error

A shared library is loaded at **a different address in each process**, so the code must be **position-independent**. An object compiled without `-fPIC` contains absolute addresses -> it can't be used to build a `.so`.

Explaining the "cannot open shared object file" Error

A direct consequence of the executable **recording only the library name**, not containing the code. Two things are commonly confused:

- `-L.` is where the **build-time linker** looks for libraries
- `ld.so` **at run time** looks somewhere else entirely: `DT_RUNPATH` embedded in the executable -> `LD_LIBRARY_PATH` -> the ldconfig cache (`/etc/ld.so.cache`) -> the default directories

```
ldd ./app     # a "not found" line is the culprit

sudo cp libmy.so.1 /usr/local/lib/ && sudo ldconfig   # install it system-wide
gcc main.c -L. -lmy -Wl,-rpath,'$ORIGIN' -o app       # embed a relative path
LD_LIBRARY_PATH=. ./app                               # ONLY for testing, not a solution
```

`LD_LIBRARY_PATH` affects every child process -> easily causes side effects. Don't use it for deployment.

Which to Choose

Situation	Choose
An ordinary application on a distro	**Shared** (gcc's default)
Security-sensitive libraries (crypto, parsers, network)	**Shared** — patchable with a single build
Rescue tools, initramfs, scratch containers	**Static**

Situation	Choose
Embedded firmware with no runtime loader	**Static**

```
ldd ./app            # which .so files are needed, and where they resolve to
file ./app           # "dynamically" or "statically linked"
LD_DEBUG=libs ./app  # see which directories ld.so tries, and in what order
nm -D ./libmy.so     # the symbols the library exports
```

`LD_DEBUG=libs` is the most effective tool when a machine has multiple copies of the same library and the program loads the wrong one.

Then I hit a problem: The shell runs. A user types a command with a bug:

```
minish$ ./buggy_program
```

The program has a `while(1) malloc(...)` loop. My machine went into swap and froze solid for 5 minutes. The shell needs a way to limit a job's resources before exec.

10. getrlimit and setrlimit Functions (Resource Limits)

```
#include <sys/resource.h>

struct rlimit {
    rlim_t rlim_cur;   /* soft limit - what the kernel actually enforces */
    rlim_t rlim_max;   /* hard limit - the ceiling for the soft limit */
};

int getrlimit(int resource, struct rlimit *rlim);
int setrlimit(int resource, const struct rlimit *rlim);
```

Three Permission Rules

- An ordinary process: free to raise/lower the **soft limit** within the hard limit.
- **The hard limit can only be lowered, never raised again** — one-way, irreversible.
- Only a privileged process (`CAP_SYS_RESOURCE`) can raise the hard limit.

Limits are inherited across `fork()` and preserved across `exec()` — this is exactly what makes the mechanism usable for a shell: `fork()` -> child calls `setrlimit()` -> child calls `exec()`, and the resulting program is constrained without **needing to know anything about it**.

`RLIM_INFINITY` = no limit.

Commonly Used Resources

Constant	Limits	What happens when exceeded
`RLIMIT_NOFILE`	Number of file descriptors	`open` / `socket` / `accept` -> `EMFILE`
`RLIMIT_AS`	Total virtual memory	`malloc` / `mmap` -> `ENOMEM`
`RLIMIT_STACK`	Stack size	Deep recursion -> `SIGSEGV`
`RLIMIT_CPU`	CPU seconds	`SIGXCPU` (soft) -> `SIGKILL` (hard)
`RLIMIT_FSIZE`	Size of files that can be created	`SIGXFSZ`, `write` -> `EFBIG`
`RLIMIT_NPROC`	Number of processes per user	`fork()` -> **`EAGAIN`**
`RLIMIT_CORE`	Core dump size	Set to 0 = disable core dumps

The corresponding shell commands: `ulimit -a` (soft), `ulimit -Ha` (hard), `ulimit -n 4096`.

Recall section 1.1: `fork()` returns `EAGAIN` — now one of its causes is clear, `RLIMIT_NPROC`.

Solving the Problem Above

```
if (pid == 0) {
    /* Constrain the job before exec */
    struct rlimit rl;
    rl.rlim_cur = rl.rlim_max = 256UL * 1024 * 1024;
    setrlimit(RLIMIT_AS, &rl);    /* 256 MB of virtual memory */

    rl.rlim_cur = rl.rlim_max = 10;
    setrlimit(RLIMIT_CPU, &rl);   /* 10 CPU seconds */

    execvp(argv[0], argv);
    _exit(127);
}
```

The memory-leaking program now gets NULL from `malloc()` after 256 MB and dies locally, instead of dragging the whole machine into swap.

Raising a Limit — The Most Common Use Case

A server needs to open 20000 sockets but the default `RLIMIT_NOFILE` is 1024 -> `accept()` returns `EMFILE`.

```
struct rlimit rl;
getrlimit(RLIMIT_NOFILE, &rl);
rl.rlim_cur = rl.rlim_max;      /* raise the soft limit UP TO the hard limit */
setrlimit(RLIMIT_NOFILE, &rl);  /* does NOT require root */
```

```
NOFILE soft=1024   hard=524288    <- before
NOFILE soft=524288 hard=524288    <- after
```

Four Side Effects

1. Lowering the hard limit is one-way. Once you lower `rlim_max`, the process **can never raise it again**. If you only want temporary self-protection, lower `rlim_cur` only and **leave** `rlim_max` **alone**.

```
getrlimit(RLIMIT_AS, &rl);
rl.rlim_cur = 256UL*1024*1024;   /* change only the soft limit */
/* rl.rlim_max untouched -> can still be raised again */
setrlimit(RLIMIT_AS, &rl);
```

2. Exceeding `RLIMIT_CPU` doesn't kill immediately. The soft limit -> `SIGXCPU` (catchable, giving a chance to clean up). Keep running to the hard limit -> `SIGKILL`, which can't be stopped.

3. `RLIMIT_STACK` relates to the `E2BIG` from section 7. The size limit on argv+environ is `RLIMIT_STACK / 4`. Lower `RLIMIT_STACK` far enough and exec starts returning `E2BIG` for command lines that aren't even that long.

4. `RLIMIT_AS` limits *virtual* memory, not RSS. A program that mmaps a large file but touches only a few pages still gets blocked, even though actual RAM usage is tiny. Recall section 8: VmSize ≠ VmRSS.

```
static void show(const char *name, int res)
{
```

```
    struct rlimit rl;
    getrlimit(res, &rl);
    printf("%-10s soft=%-12llu hard=%llu\n", name,
           (unsigned long long) rl.rlim_cur, (unsigned long long) rl.rlim_max);
}
```

11. ELF, Symbol Table, Relocation, and Linking

11.1 What ELF Is

ELF (Executable and Linkable Format) is the standard file format on Linux, shared by several kinds of file:

File	Role
`main.o`	Relocatable object — the compiler's output
`program`	Executable
`libmath.so`	Shared object
`libmath.a`	**Not ELF** — just an archive containing several `.o` files

ELF is neither a "static format" nor a "dynamic format". It is the container for the information needed by the **linker** (at build time) and the **loader** (at run time).

Main structure:

```
ELF
├── ELF header
├── Program headers      <- information for the loader
├── Section headers      <- information for the linker
└── Sections
    ├── .text            <- machine code
    ├── .data            <- variables initialized to nonzero
    ├── .bss             <- variables initialized to zero
    ├── .symtab          <- symbol table
    ├── .strtab          <- symbol names
    └── .rela.text       <- relocations for .text
```

11.2 Symbols and the Symbol Table

A **symbol** is a name meaningful to the linker: a function name, a global variable name.

The **symbol table** is a directory mapping `name -> information (type, section, value)`.

```
/* main.c */
int add(int a, int b);
int main(void) { return add(2, 3); }
```

Symbol table of `main.o`:

```
Symbol    Section    Value
-------------------------
main      .text      0x00
add       UND        0x00
```

`UND` (undefined) means: *this file uses `add`, but the definition lives elsewhere.*

The compiler processes **one `.c` file at a time**, so it cannot know the address of anything outside that file. It records the name and leaves the slot empty.

11.3 Relocation

When the compiler doesn't know the address of `add`, the generated machine code has a hole:

```
call ????????
```

At the same time the compiler writes an entry into the **relocation table** (`.rela.text`):

```
offset      symbol
--------------------
0x...       add
```

Meaning: "*at this byte offset in `.text`, fill in the address of the symbol `add`.*"

Once the linker knows `add = 0x401126`, it patches:

```
call ????????   ->   call 0x401126
```

Two concepts that are often conflated:

- **Symbol resolution:** answers "*where is this symbol defined?*"
- **Relocation:** "*patch code/data to point at that address.*" The `.o` files inside a static `.a` library **still carry** relocations — they are ordinary object files that have not been processed yet.

11.4 Static Linking

Input: `main.o` + `libmath.a`.

`libmath.a` is an archive containing `add.o`, `sub.o`, `multiply.o`... The linker **extracts only the `.o` files it needs** to resolve the `UND` symbols, then:

```
resolve symbols
      ↓
assign final addresses
      ↓
apply relocations
      ↓
complete executable
```

The code of `add()` is **copied into the executable**:

```
program
.text
+----------------+
| main           |
| add            |  <- copied from add.o
+----------------+
```

After this step `add` is no longer `UND`. The program runs without needing any library.

11.5 Dynamic Linking

Input: `main.o` + `libmath.so`.

The linker still checks "*does `libmath.so` provide `add`?*" but **does not copy the code into the executable**. Instead it records a dependency:

```
Dynamic section:
  NEEDED    Shared library: [libmath.so]
  NEEDED    Shared library: [libc.so.6]
```

The executable says: "*I don't contain `add()`. At run time, load `libmath.so`.*"

At run time, the **dynamic loader** (`ld.so`) does the rest:

```
executable -> dynamic loader -> load .so -> resolve symbols -> apply runtime relocations ->
run
```

11.6 .symtab and .dynsym

Because dynamic linking must look symbol names up **at run time**, ELF carries two separate symbol tables:

Table	Serves	Note
`.symtab`	linker, debugger, inspection	Complete; removed by `strip`
`.dynsym`	dynamic linking at run time	Only exported/imported symbols; **not stripped**

11.7 GOT and PLT

The dynamic loader does not patch individual instructions in `.text` the way the static linker does. Instead, function calls go through two intermediate tables:

- **PLT** (Procedure Linkage Table): trampoline stubs in the code region
- **GOT** (Global Offset Table): a table of pointers in the writable data region

```
main
 | call add@PLT
 ▼
PLT ──► GOT ──► libmath.so : add
```

Initially `GOT[add]` holds no real address. On the first call, the loader's resolver looks the symbol up and writes:

```
GOT[add] = the real address of add
```

Subsequent calls go straight through the resolved GOT entry.

This mechanism exists because `.text` must stay read-only and shareable between processes, so everything that needs patching is pushed into the GOT.

11.8 .a vs .so

	`libmath.a`	`libmath.so`
Nature	Archive of several `.o` files	A complete ELF shared object
What the linker does	Extracts the needed `.o`, embeds it in the executable	Only records the dependency
At run time	Not required	Must be present
Multiple processes	One copy per process	Shared

11.9 Overall Comparison

```
STATIC                          DYNAMIC
source                          source
  ↓ compiler                      ↓ compiler
 .o                              .o
  ↓ linker                        ↓ linker
executable (all code)           executable + DT_NEEDED
  ↓                               ↓
RUN                             dynamic loader
                                  ↓ load .so
                                  ↓ resolve symbols
                                  ↓ runtime relocation
                                RUN
```

Concept	Static	Dynamic
Symbol resolution	Build time	Build time + run time
Relocation	Linker	Linker + dynamic loader
`.dynsym`	Unimportant	Essential
`DT_NEEDED`	Absent	Present
GOT / PLT	Not needed	Required
Library code in the executable	Yes	No
Library needed at run time	No	Yes
Dynamic loader	No	Yes

12. Stack Frame Layout — System V AMD64 ABI

Scope: Linux/BSD/macOS on x86-64. Windows x64 and ARM AAPCS differ — see the last subsection.

12.1 Why It's Needed

Every function call needs its own set of storage slots: parameters, local variables, return address. Registers are neither numerous enough nor duplicable across nested calls. A stack frame is the slice of memory allocated for one call and released when the function returns.

An ABI is the binary contract between caller and callee: who puts what where, and who is responsible for cleaning up. Violating the contract crashes somewhere unrelated to the actual bug.

12.2 Core Rules

Registers

	Register	Note
Stack pointer	`%rsp`	Points at the top of the stack (the lowest address in use). The hardware adjusts it via `push` / `pop` / `call` / `ret`
Frame pointer	`%rbp`	A fixed anchor for the frame. **A convention, not a requirement**

The stack grows downward: allocate = `sub $N,%rsp`, release = `add $N,%rsp`.

Parameter Passing

Kind	Location
Integer/pointer #1–6	`%rdi` `%rsi` `%rdx` `%rcx` `%r8` `%r9`
Float/double #1–8	`%xmm0` – `%xmm7`
The rest	Stack, pushed by the caller **right to left**

The two sequences count **independently**: `f(int a, double b, int c)` -> `a` = `rdi`, `c` = `rsi`, `b` = `xmm0`.

For variadic functions, `%al` holds the number of xmm registers used as arguments. That's why `xor %eax,%eax` appears before `call printf`.

Return Values

Type	Returned in
Integer/pointer	`%rax`
Struct ≤ 16 bytes	`%rax:%rdx`
float/double	`%xmm0` (`:%xmm1`)
Struct > 16 bytes	The caller allocates the memory and passes a hidden address in `%rdi` (shifting the real arguments one slot back); the callee returns that same address in `%rax`

Preservation Responsibility

Group	Registers
Callee-saved (the callee must return them intact)	`rbx` `rbp` `r12` `r13` `r14` `r15` `rsp`
Caller-saved (volatile across `call`)	`rax` `rcx` `rdx` `rsi` `rdi` `r8`–`r11`, all `xmm`

Consequence: `push %rbx` at the top of a function is **saving a callee-saved register**, not a frame-pointer prologue.

Alignment

`%rsp` must be **a multiple of 16 at the moment the `call` instruction executes**.

It follows that on entry to the callee `%rsp % 16 == 8` (because `call` pushed 8 bytes), and after `push %rbp` that `%rbp % 16 == 0`.

12.3 Prologue / Epilogue

```
; PROLOGUE
push  %rbp          ; save the caller's anchor
mov   %rsp, %rbp    ; set the new anchor; from here %rbp stays put
sub   $N, %rsp      ; allocate N bytes of locals + spill area

; EPILOGUE
leave               ; = mov %rbp,%rsp ; pop %rbp
ret                 ; = pop %rip
```

No `sub` -> only `pop %rbp; ret` is needed.

Other callee-saved registers are `push`ed **after** `mov %rsp,%rbp`, and `pop`ped in reverse order before `leave`.

12.4 Frame Map

```
  HIGH
+----------------------------+
|  arg 8                     | +24(%rbp)  ┐ CALLER's frame
|  arg 7                     | +16(%rbp)  ┘ the callee only reads it
+----------------------------+
|  return address            |  +8(%rbp)   pushed by the CALL instruction
+----------------------------+
|  saved %rbp                |  +0(%rbp)   from "push %rbp"
+============================+ === %rbp, multiple of 16 ===
|  pushed callee-saved regs  |            ┐
|  local variables           | -8, -16... │ OWN FRAME
|  register-parameter spill  |            ┘ (= the N in sub $N)
+============================+ === %rsp ===
|  RED ZONE, 128 bytes       |
+----------------------------+
  LOW        (the stack grows downward)
```

Three regions to keep distinct: **positive offsets ≥ 16** = stack parameters, belonging to the caller; **+8/+0** = infrastructure; **negative offsets** = the function's own frame.

Local arrays grow **upward**: `arr[0]` is at the lowest address, `arr[n-1]` at the highest — the opposite direction from the stack.

12.5 The Red Zone (128 Bytes)

Definition (AMD64 ABI): the 128 bytes below `%rsp` are reserved, and signal handlers and interrupt handlers must not modify them. A function uses this area for temporary data that need not survive a function call. A leaf function can use it as its entire frame, never touching `%rsp`.

Benefit: exactly two instructions saved (`sub` + `add`).

Condition: usable only in a **leaf function** (one that calls nothing). Any `call` writes a return address on top of the red zone.

How to spot it in a disassembly: stack accesses at negative offsets from `%rsp` with **no** `sub` instruction.

```
; -O1              (red zone)      | ; -O1 -mno-red-zone
leaf:                              | leaf:
  mov %rdi,-0x28(%rsp)   <- neg    |   sub $0x28,%rsp     <- appears
  mov %rsi,-0x20(%rsp)             |   mov %rdi,(%rsp)    <- positive
  ret                              |   add $0x28,%rsp
                                   |   ret
```

`-mno-red-zone` **is mandatory** for kernel modules, code running in interrupt context, and bare-metal x86. The CPU pushes interrupt state immediately below `%rsp` and knows nothing about the red zone. Forget this flag and you get random stack corruption that only happens when an interrupt lands at exactly the wrong moment — not reproducible.

12.6 Omitting the Frame Pointer

The ABI states plainly that `%rbp` is optional: indexing into the frame via `%rsp` saves two instructions and frees one general-purpose register.

gcc **omits** `%rbp` **by default** on x86-64 when optimization is on. Keep it with `-fno-omit-frame-pointer`.

	With `%rbp`	**Without** `%rbp`
Variable offsets	Fixed throughout the function	**Change with every** `push` / `pop` / `sub`
Stack trace	Follow `%rbp` like a linked list	Requires CFI in `.eh_frame` (DWARF)
Reading by eye	Easy	You must track `%rsp` manually

The same 8-parameter function at `-O2`: no `push %rbp`, no `sub`, stack parameters read via `0x8(%rsp)` / `0x10(%rsp)` — and those offsets **shift by 8** after every `push`.

12.7 Reading Assembly: A Six-Step Procedure

Tools:

```
gcc -S -O0 -fno-omit-frame-pointer -fno-stack-protector -fverbose-asm f.c
objdump -d --no-show-raw-insn f.o     # already compiled
objdump -dS f.o                       # interleaved source (needs -g)
readelf -wf f.o                       # read the CFI when there's no %rbp
```

1. **Identify the function boundaries** — `<name>:` … `ret`, or `.size name, .-name`.

2. **Classify the prologue:** | What you see | Conclusion | |---|---| | `push %rbp` + `mov %rsp,%rbp` | Frame pointer present | | Only `sub $N,%rsp` | No frame pointer, frame of N bytes | | Nothing at all | Leaf function using the red zone, or no stack needed | | `push %rbx`/`%r12`… | Callee-saved, **not** a frame pointer |
3. **Compute the frame size** = `N` + 8 × the number of `push`es in the prologue. No `sub` -> use `readelf -wf` and read `DW_CFA_def_cfa_offset`.
4. **Classify each memory access** using the table in 12.4.
5. **Look for** `call` in the body — none -> leaf function -> allowed to use the red zone.
6. **Cross-check the epilogue** — every `push` has a `pop` in reverse order, every `sub $N` has an `add $N` or a `leave`. A mismatch means you got the function boundaries wrong.

Labs

Lab 1. Fork/exec/wait/waitpid; observe zombie and orphan processes

Link Git Lab Fork/Exec

Lab 2. Race condition between fork and exec

Link Git Lab Race Condition

Lab 3. Use ps -o pid,ppid,pgid,sid to observe the process

group/session

Observing Process Groups and Sessions base on Lab 2

Beyond the file descriptor leak, this example is also useful for observing how `fork()` and `exec()` affect a process's **process group** and **session**.

The four fields

Field	Meaning
`pid`	Process ID of the process itself.
`ppid`	Parent PID — which process called `fork()` to create it.
`pgid`	Process Group ID — signals such as `SIGINT` from `Ctrl+C` are delivered to the whole group.
`sid`	Session ID — normally tied to a controlling terminal.

A process where `pid == pgid` is a **process group leader**. A process where `pid == sid` is a **session leader**.

Keeping the child alive long enough to observe

`show_fds` normally exits within a few milliseconds, which is too fast for `ps` to catch. Add a delay at the end of `show_fds.c` (requires `#include <unistd.h>`):

```
printf("[show_fds] pid=%d pgid=%d sid=%d - sleeping 30s\n",
       getpid(), getpgrp(), getsid(0));
sleep(30);
```

`sleep()` is preferred over `getchar()` here, because the parent and the child would otherwise compete for the same stdin.

Recompile:

```
gcc -Wall -Wextra show_fds.c -o show_fds
```

Running the observation

Start the program in the background, then inspect the process table:

```
./race &
sleep 1
ps -o pid,ppid,pgid,sid,comm -e | grep -Ew 'race|show_fds|bash'
```

The `-w` flag of `grep` matches whole words only. Without it, unrelated kernel threads such as `rcu_tasks_trace_kthread` also match the string `race`.

Example output

```
  PID   PPID   PGID    SID  COMMAND
 9804   4517   9804   9804  bash
11530   9804  11530   9804  race
11538  11530  11530   9804  show_fds
```

What the output shows

Parent-child chain. `show_fds` (11538) has `race` (11530) as its parent, which in turn has `bash` (9804) as its parent. Note that 11538 is the PID assigned at `fork()` time: `exec()` replaces the program image but does **not** create a new process, so the PID is unchanged.

Process group is inherited. Both `race` and `show_fds` share `PGID = 11530`. The child does not start a new group. Because `race` has `PID == PGID`, it is the group leader — the shell placed it in its own job when it was started with `./race &`. As a result, a signal sent to the group (`kill -TERM -11530`, or `Ctrl+C` for a foreground job) reaches both processes.

Session is inherited. All three processes share `SID = 9804`, which equals the PID of `bash`. `bash` is therefore the session leader, and the session is attached to the current terminal. Neither `fork()` nor `exec()` changes the session ID — only `setsid()` creates a new one.

Resulting hierarchy

```
Session 9804  (leader: bash, attached to the terminal)
|
+-- Process group 9804    -> bash
|
+-- Process group 11530   -> race      (group leader, PID == PGID)
                          -> show_fds  (inherits PGID)
```

Lab 4. Write a program that prints the addresses of its own memory regions (text/data/bss/heap/stack)

Link Git Lab Memory

Lab 5. Use nm/readelf/ldd/objdump to inspect a self-written binary and produce a short report with project static and dynamic

nm

```
trung@trungnm:~/Documents/session3/static and dynamic/static$ nm main_static
000000000000038c r __abi_tag
000000000000117d T add
0000000000004010 B __bss_start
0000000000004010 b completed.0
                 w __cxa_finalize@GLIBC_2.2.5
0000000000004000 D __data_start
0000000000004000 W data_start
0000000000001090 t deregister_tm_clones
0000000000001100 t __do_global_dtors_aux
0000000000003dc0 d __do_global_dtors_aux_fini_array_entry
0000000000004008 D __dso_handle
0000000000003dc8 d _DYNAMIC
0000000000004010 D _edata
0000000000004018 B _end
0000000000001198 T _fini
0000000000001140 t frame_dummy
0000000000003db8 d __frame_dummy_init_array_entry
0000000000002118 r __FRAME_END__
0000000000003fb8 d _GLOBAL_OFFSET_TABLE_
                 w __gmon_start__
0000000000002010 r __GNU_EH_FRAME_HDR
0000000000001000 T _init
0000000000002000 R _IO_stdin_used
                 w _ITM_deregisterTMCloneTable
                 w _ITM_registerTMCloneTable
                 U __libc_start_main@GLIBC_2.34
0000000000001149 T main
                 U printf@GLIBC_2.2.5
00000000000010c0 t register_tm_clones
```

readelf

readelf -d main_dynamic

```
trung@trungnm:~/Documents/session3/static and dynamic/dynamic$ readelf -d main_d
ynamic

Dynamic section at offset 0x2db0 contains 28 entries:
  Tag        Type                         Name/Value
 0x0000000000000001 (NEEDED)             Shared library: [libmath.so]
 0x0000000000000001 (NEEDED)             Shared library: [libc.so.6]
 0x000000000000000c (INIT)               0x1000
 0x000000000000000d (FINI)               0x11a0
 0x0000000000000019 (INIT_ARRAY)         0x3da0
 0x000000000000001b (INIT_ARRAYSZ)       8 (bytes)
 0x000000000000001a (FINI_ARRAY)         0x3da8
 0x000000000000001c (FINI_ARRAYSZ)       8 (bytes)
 0x000000006ffffef5 (GNU_HASH)           0x3b0
 0x0000000000000005 (STRTAB)             0x498
 0x0000000000000006 (SYMTAB)             0x3d8
 0x000000000000000a (STRSZ)              158 (bytes)
 0x000000000000000b (SYMENT)             24 (bytes)
 0x0000000000000015 (DEBUG)              0x0
 0x0000000000000003 (PLTGOT)             0x3fb0
 0x0000000000000002 (PLTRELSZ)           48 (bytes)
 0x0000000000000014 (PLTREL)             RELA
 0x0000000000000017 (JMPREL)             0x638
```

readelf --dyn-syms main_dynamic

```
trung@trungnm:~/Documents/session3/static and dynamic/dynamic$ readelf --dyn-sym
s main_dynamic

Symbol table '.dynsym' contains 8 entries:
   Num:    Value          Size Type    Bind   Vis      Ndx Name
     0: 0000000000000000     0 NOTYPE  LOCAL  DEFAULT  UND
     1: 0000000000000000     0 FUNC    GLOBAL DEFAULT  UND _[...]@GLIBC_2.34 (2)
     2: 0000000000000000     0 NOTYPE  WEAK   DEFAULT  UND _ITM_deregisterT[...]
     3: 0000000000000000     0 FUNC    GLOBAL DEFAULT  UND add
     4: 0000000000000000     0 FUNC    GLOBAL DEFAULT  UND [...]@GLIBC_2.2.5 (3)
     5: 0000000000000000     0 NOTYPE  WEAK   DEFAULT  UND __gmon_start__
     6: 0000000000000000     0 NOTYPE  WEAK   DEFAULT  UND _ITM_registerTMC[...]
     7: 0000000000000000     0 FUNC    WEAK   DEFAULT  UND [...]@GLIBC_2.2.5 (3)
trung@trungnm:~/Documents/session3/static and dynamic/dynamic$
```

readelf -r main_dynamic

```
trung@trungnm:~/Documents/session3/static and dynamic/dynamic$ readelf -r main_d
ynamic

Relocation section '.rela.dyn' at offset 0x578 contains 8 entries:
  Offset          Info           Type           Sym. Value    Sym. Name + Addend
000000003da0  000000000008 R_X86_64_RELATIVE                    1160
000000003da8  000000000008 R_X86_64_RELATIVE                    1120
000000004008  000000000008 R_X86_64_RELATIVE                    4008
000000003fd8  000100000006 R_X86_64_GLOB_DAT 0000000000000000 __libc_start_main@
GLIBC_2.34 + 0
000000003fe0  000200000006 R_X86_64_GLOB_DAT 0000000000000000 _ITM_deregisterTM[
...] + 0
000000003fe8  000500000006 R_X86_64_GLOB_DAT 0000000000000000 __gmon_start__ + 0
000000003ff0  000600000006 R_X86_64_GLOB_DAT 0000000000000000 _ITM_registerTMCl[
...] + 0
000000003ff8  000700000006 R_X86_64_GLOB_DAT 0000000000000000 __cxa_finalize@GLI
BC_2.2.5 + 0

Relocation section '.rela.plt' at offset 0x638 contains 2 entries:
  Offset          Info           Type           Sym. Value    Sym. Name + Addend
000000003fc8  000300000007 R_X86_64_JUMP_SLO 0000000000000000 add + 0
000000003fd0  000400000007 R_X86_64_JUMP_SLO 0000000000000000 printf@GLIBC_2.2.5
 + 0
```

ldd

```
trung@trungnm:~/Documents/session3/static and dynamic/dynamic$ export LD_LIBRARY
_PATH=.
trung@trungnm:~/Documents/session3/static and dynamic/dynamic$ ldd main_dynamic
        linux-vdso.so.1 (0x0000785329d75000)
        libmath.so => ./libmath.so (0x0000785329d5f000)
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x0000785329a00000)
        /lib64/ld-linux-x86-64.so.2 (0x0000785329d77000)
```

objdump

```
trung@trungnm:~/Documents/session3/static and dynamic/dynamic$ objdump -d main_dynamic

main_dynamic:     file format elf64-x86-64


Disassembly of section .init:

0000000000001000 <_init>:
    1000:       f3 0f 1e fa             endbr64
    1004:       48 83 ec 08             sub    $0x8,%rsp
    1008:       48 8b 05 d9 2f 00 00    mov    0x2fd9(%rip),%rax        # 3fe8 <__gmon_
start__@Base>
    100f:       48 85 c0                test   %rax,%rax
    1012:       74 02                   je     1016 <_init+0x16>
    1014:       ff d0                   call   *%rax
    1016:       48 83 c4 08             add    $0x8,%rsp
    101a:       c3                      ret

Disassembly of section .plt:

0000000000001020 <.plt>:
    1020:       ff 35 92 2f 00 00       push   0x2f92(%rip)        # 3fb8 <_GLOBAL_OFFS
ET_TABLE_+0x8>
    1026:       ff 25 94 2f 00 00       jmp    *0x2f94(%rip)        # 3fc0 <_GLOBAL_OFF
SET_TABLE_+0x10>
    102c:       0f 1f 40 00             nopl   0x0(%rax)
    1030:       f3 0f 1e fa             endbr64
    1034:       68 00 00 00 00          push   $0x0
    1039:       e9 e2 ff ff ff          jmp    1020 <_init+0x20>
    103e:       66 90                   xchg   %ax,%ax
```

Revision #12
Created 2026-08-25 10:14:01 UTC by trungnguyen
Updated 2026-09-03 03:04:14 UTC by trungnguyen